CENTRO UNIVERSITÁRIO MÁRIO PALMÉRIO
CURSO CIENCIA DA COMPUTAÇÃO

Henry Miller de Barros Malagoli

TRABALHO III: EXPLORANDO O PODER DA REUTILIZAÇÃO COM GENERICS EM JAVA

Monte Carmelo
2026
Henry Miller de Barros Malagoli

Parte I: Investigação Teórica

1. O Problema Original: O Mundo antes dos Generics

1.a) Funcionamento das coleções antes do Java 5

Antes do lançamento do Java 5 (JDK 1.5, em 2004), o *Java Collections Framework* (como `ArrayList`, `LinkedList`, `HashSet` e `HashMap`) não possuía parametrização de tipos (Oracle Java Tutorials - Why Use Generics?). Isso significava que todas as estruturas de dados operavam de forma genérica apenas através de polimorfismo de subtipagem, armazenando internamente referências do tipo mais geral da linguagem: a classe `java.lang.Object`. Ao declarar um `ArrayList`, o compilador não tinha nenhuma informação sobre se aquela lista deveria conter apenas objetos `String`, `Integer` ou `Produto`.

1.b) O papel da classe Object no armazenamento

Em Java, todas as classes herdam direta ou indiretamente de `Object` (Oracle Java Tutorials - Generic Types). Por causa dessa herança universal, um método como `add(Object o)` permitia inserir qualquer instância dentro da mesma coleção. Embora isso garantisse reutilização de código (não era necessário criar uma classe `ArrayListDeStrings` e outra `ArrayListDeProdutos`), o retorno do método `get(int index)` também era do tipo `Object`. Assim, sempre que o programador precisava recuperar um elemento e usar métodos específicos da sua classe real, era obrigatório realizar uma conversão explícita de tipo (*downcasting*).

1.c) Exemplo executável provocando ClassCastException

O código abaixo demonstra o problema clássico de uma coleção sem Generics (usando *raw type*, como era feito antes do Java 5), onde misturamos uma `String` e um `Integer` na mesma lista e tentamos converter todos os elementos para `String` na leitura:

```
import java.util.ArrayList;
import java.util.List;

public class ExemploAntesDosGenerics {
    @SuppressWarnings({"rawtypes", "unchecked"})
    public static void main(String[] args) {
        List listaProdutos = new ArrayList();
        listaProdutos.add("Notebook Dell");
        listaProdutos.add(12345); // Inserindo um Integer por engano

        for (int i = 0; i < listaProdutos.size(); i++) {
            // Cast explicito obrigatorio; falha no indice 1
            String nome = (String) listaProdutos.get(i);
            System.out.println("Produto: " + nome.toUpperCase());
        }
    }
}
```

Saída real gerada no console ao executar o programa:

```
Produto: NOTEBOOK DELL
Exception in thread "main" java.lang.ClassCastException: class
java.lang.Integer cannot be cast to class java.lang.String
(java.lang.Integer and java.lang.String are in module java.base of loader
'bootstrap')
    at ExemploAntesDosGenerics.main(ExemploAntesDosGenerics.java:12)
```

1.d) Perigos dessa abordagem e Segurança de Tipo (Type Safety)

O principal perigo do uso direto de `Object` é a quebra da **segurança de tipo (*type safety*)** (Oracle Java Tutorials - Why Use Generics?; Bloch, *Effective Java*, Item 26). Como o compilador aceita qualquer objeto no método `add(Object)`, erros de lógica cometidos pelo desenvolvedor — como guardar um `Alimento` em uma lista destinada a `Eletronico` — passam pela compilação sem nenhum erro. O defeito fica oculto até que o fluxo de execução passe exatamente pela linha do *cast* inválido, provocando um `ClassCastException` em produção, muitas vezes longe do ponto onde o objeto incorreto foi inserido.

2. A Solução: O que são e para que servem os Generics?

2.a) Definição de Generics e convenção dos parâmetros de tipo

Generics (tipos parametrizados), introduzidos no Java 5, permitem que classes, interfaces e métodos recebam tipos como parâmetros no momento da declaração e instanciação (Oracle Java Tutorials - Generic Types). Dessa forma, é possível reutilizar o mesmo código com diferentes tipos de dados mantendo a

verificação estática. Pela convenção oficial do Java, usam-se letras maiúsculas únicas para representar os parâmetros formais de tipo:

Símbolo	Significado	Uso Comum no Java
`<T>`	*Type* (Tipo)	Representa um tipo genérico qualquer em classes utilitárias ou de domínio (ex.: `Deposito<T>` , `Optional<T>`).
`<E>`	*Element* (Elemento)	Usado nas coleções do pacote `java.util` para indicar os elementos guardados (ex.: `List<E>` , `Set<E>`).
`<K>`	*Key* (Chave)	Representa o tipo da chave em estruturas associativas ou dicionários (ex.: `Map<K, V>`).
`<V>`	*Value* (Valor)	Representa o tipo do valor associado a uma chave em mapas (ex.: `Map<K, V>`).

2.b) Como os Generics resolvem o problema de Type Safety

Com Generics, o programador informa explicitamente ao compilador qual tipo a estrutura aceita, por exemplo `List<Eletronico>` . A partir disso, o compilador substitui a checagem tardia em tempo de execução por uma **verificação estática em tempo de compilação** (Oracle Java Tutorials - Why Use Generics?). Se alguém tentar chamar `lista.add(new Alimento(...))` , o código sequer compila e a IDE (como o IntelliJ IDEA) aponta o erro imediatamente. Além disso, o compilador sabe exatamente o tipo retornado na leitura, eliminando a necessidade de *casts* manuais e evitando `ClassCastException` .

2.c) Sintaxe básica: Classe Genérica e Método Genérico

Na declaração de uma **classe genérica**, o parâmetro de tipo `<T>` é colocado logo após o nome da classe. Já em um **método genérico**, o parâmetro `<T>` deve ser declarado antes do tipo de retorno do método (Oracle Java Tutorials - Generic Methods):

```
// Exemplo de Classe Generica
public class Caixa<T> {
    private T conteudo;

    public void guardar(T item) { this.conteudo = item; }
    public T abrir() { return this.conteudo; }
}

// Exemplo de Metodo Generico independente
public class Utilitario {
    public static <T> T obterPrimeiro(java.util.List<T> lista) {
        return lista.isEmpty() ? null : lista.get(0);
    }
}
```

2.d) Evolução da sintaxe (Java 7 e Java 10) e versão do JDK utilizada Java 5 (2004):

- Exigia repetir o tipo na declaração e no construtor: `Deposito<Eletronico> dep = new Deposito<Eletronico>("Setor A");` .
- **Diamond Operator <> (Java 7):** Permitiu omitir o tipo no construtor, pois o compilador infere o argumento pelo lado esquerdo da atribuição (Oracle Java Tutorials - The Diamond): `Deposito<Eletronico> dep = new Deposito<>("Setor A");` .
- **Inferência de variáveis locais com `var` (Java 10):** Permitiu inferir o tipo da variável local a partir do construtor parametrizado no lado direito: `var dep = new Deposito<Eletronico>("Setor A");` .
- **Versão utilizada neste trabalho: Java JDK 17** (OpenJDK 17), compilado e executado pelo IntelliJ IDEA.

3. Explorando mais a Fundo

3.a) Type Erasure (Apagamento de Tipo), consequências práticas e Raw Types

O **Type Erasure** é o mecanismo pelo qual o compilador Java implementa Generics garantindo compatibilidade binária com versões antigas da JVM (Oracle Java Tutorials Type Erasure). Durante a compilação, o compilador valida todas as regras de tipos, mas em seguida **apaga** os parâmetros genéricos do bytecode gerado: parâmetros ilimitados (como `<T>`) viram `Object` , e parâmetros limitados (como `<T extends Produto>`) são substituídos pelo seu primeiro limite (`Produto`). Quando necessário, o próprio compilador insere *casts* sintéticos seguros no bytecode.

Duas consequências práticas diretas do Type Erasure:

- **Impossibilidade de instanciar `new T()` :** Como `T` é apagado para `Object` (ou seu limite) em tempo de execução, a JVM não sabe qual construtor concreto chamar dentro da classe genérica.

⁉️ **Impossibilidade de usar `instanceof List<String>`:** Em tempo de execução, tanto `List<String>` quanto `List<Integer>` possuem exatamente a mesma classe `ArrayList.class`, sendo permitido verificar apenas `obj instanceof List<?>`.

O risco de Raw Types: Usar uma classe genérica sem passar seus parâmetros (ex.: `Deposito dep = new Deposito("Geral")`) é chamado de *raw type* (Oracle Java Tutorials - Raw Types). Os *raw types* só existem por legado e desativam a checagem estática dos Generics, emitindo avisos de `unchecked call` e voltando a expor o sistema ao `ClassCastException`.

3.b) Bounded Types (Tipos Limitados)

Os **bounded types** restringem quais tipos podem ser usados como argumento de um parâmetro genérico, utilizando a palavra-chave `extends` (Oracle Java Tutorials - Bounded Type Parameters). Além de impedir tipos indesejados, isso permite invocar métodos definidos na superclasse ou interface limite diretamente sobre a variável `T`. Por exemplo, em `<T extends Comparable<T>>`, garantimos que qualquer tipo `T` possua o método `compareTo`:

```
public static <T extends Comparable<T>> T maiorElemento(T a, T b) {
    return (a.compareTo(b) >= 0) ? a : b; // Chamada segura a compareTo()
}
```

3.c) Invariância dos Generics vs. Covariância dos Arrays

Em Java, os **arrays são covariantes**: como `String` é subtipo de `Object`, o tipo `String[]` é considerado subtipo de `Object[]`. Já os **Generics são invariantes**: mesmo que `String` herde de `Object` (ou `Eletronico` herde de `Produto`), `List<String>` **não é** subtipo de `List<Object>` (Bloch, *Effective Java*, Item 28).

Essa decisão existe para preservar a segurança em tempo de compilação. Nos arrays, a covariância permite que o código compile, mas falhe em execução com `ArrayStoreException`:

```
// Arrays sao covariantes (compila, mas lanca ArrayStoreException em
execucao):
Object[] vetor = new String[2];
vetor[0] = 100; // Erro em tempo de execucao!

// Generics sao invariantes (nao compila, protegendo o desenvolvedor):
// List<Object> lista = new ArrayList<String>(); // Erro de compilacao!
```

3.d) Wildcards (`<?>`, `<? extends T>`, `<? super T>`) e a Regra PECS

Para contornar a rigidez da invariância quando criamos métodos que operam sobre coleções ou depósitos, utilizamos os **wildcards** (coringas representados por `?`) (Oracle Java Tutorials - Wildcards). No livro *Effective Java* (Item 31), Joshua Bloch definiu a regra mnemônica **PECS (*Producer Extends, Consumer Super*)** para escolher o coringa correto:

- **Unbounded Wildcard (`<?>`):** Representa um tipo desconhecido qualquer (Oracle Unbounded Wildcards). É útil quando o método apenas lê os elementos tratando-os como `Object` (ou pelo limite base da classe, como `Produto`) ou usa métodos que não dependem do parâmetro de tipo (ex.: `size()`, `isEmpty()`). Exemplo: `listarConteudo(Deposito<?> deposito)`.
- **Upper Bounded Wildcard (`<? extends Tipo>`) — *Producer Extends*:** Aceita `Tipo` ou qualquer subclasse dele (Oracle - Upper Bounded Wildcards). Deve ser usado quando a estrutura atua como **produtora** de dados (nós apenas lemos/retiramos itens dela).
  Exemplo: ler itens de um `Deposito<? extends Produto>`, aceitando tanto `Deposito<Eletronico>` quanto `Deposito<Alimento>`.
- **Lower Bounded Wildcard (`<? super Tipo>`) — *Consumer Super*:** Aceita `Tipo` ou qualquer superclasse dele (Oracle - Lower Bounded Wildcards). Deve ser usado quando a estrutura atua como **consumidora** (nós inserimos itens do tipo `Tipo` nela). Exemplo: transferir objetos `Eletronico` para um `Deposito<? super Eletronico>`, o que permite guardá-los tanto em um `Deposito<Eletronico>` quanto em um `Deposito<Produto>` geral.

Parte II: Aplicação Prática e Evidências

1. Refatoração da Hierarquia e da Classe Deposito

Para resolver as falhas do código original da empresa de logística, estruturamos primeiramente a superclasse abstrata `Produto` (contendo `nome` e `preco`) e fizemos as classes `Eletronico` (com `garantiaMeses`), `Alimento` (com `dataValidade`) e `Vestuario` (com `tamanho`) herdarem de `Produto`.

Em seguida, refatoramos a classe `Deposito` para utilizar um parâmetro genérico limitado: `public class Deposito<T extends Produto>`. Com essa alteração:

Um `Deposito<Eletronico>` só aceita instâncias de `Eletronico` no método `armazenar(T item)`.

Qualquer tentativa de criar um depósito com tipos fora da hierarquia (como `Deposito<String>` ou `Deposito<Integer>`) é barrada pelo compilador por violar a restrição `extends Produto`.

- O método `retirar()` passou a retornar `Optional<T>` em vez de `null` quando o depósito está vazio, evitando ocorrências de `NullPointerException` e obrigando quem chama o método a tratar a ausência de itens de forma segura.

2. Evidências na Main: Antes e Depois da Refatoração

2.a) Comparativo "Antes e Depois" (Erro em Execução vs. Segurança em Compilação)

No código problemático original (sem Generics), a classe `Deposito` armazenava uma lista de `Object` (ou `Produto` sem parametrização). Quando um operador guardava por engano um `Alimento` no depósito de eletrônicos, o código compilava normalmente, mas quebrava durante a execução ao fazer o *cast* para `Eletronico` :

```
// ANTES (Codigo problematico sem Generics): DepositoAntigo
depEletronicos = new DepositoAntigo(); depEletronicos.armazenar(new
Eletronico("Notebook Dell", 4200.00, 12)); depEletronicos.armazenar(new
Alimento("Feijao Carioca 1kg", 8.90, "20/10/2027"));

// Na retirada, o cast (Eletronico) falhava em tempo de execucao:
Eletronico item = (Eletronico) depEletronicos.retirar();
```

Saída do erro em tempo de execução no código antigo:

```
Exception in thread "main" java.lang.ClassCastException: class Alimento
cannot be cast to class Eletronico (Alimento and Eletronico are in unnamed
module of loader 'app')
    at MainAntiga.main(MainAntiga.java:10)
```

2.b) Evidência do erro de compilação na nova versão genérica (Tarefa 2.c)

Na versão refatorada com `Deposito<Eletronico>` , ao descomentar a linha na `Main.ja` armazenar um `Alimento` dentro de `depEletronicos` , o compilador do Java (tanto no IntelliJ IDEA quanto via `javac`) interrompe o build imediatamente com a seguinte mensagem:

```
// Trecho testado na Main.java: depEletronicos.armazenar(new
Alimento("Feijão Carioca 1kg", 8.90, "20/10/2027"));
```

```
Main.java:18: error: incompatible types: Alimento cannot be converted to
Eletronico         depEletronicos.armazenar(new Alimento("Feijão Carioca
1kg", 8.90,
"20/10/2027"));
                                   ^
Note: Some messages have been simplified; recompile with -Xdiags:verbose to
get full output
1 error
```

2.c) Retirada sem necessidade de Cast e com Optional (Tarefa 2.d)

Ao retirar um item de `depEletronicos`, o retorno já vem tipado como `Optional<Eletronico>`, permitindo acessar diretamente métodos específicos como `getGarantiaMeses()` sem nenhum `(Eletronico)`:

```
Optional<Eletronico> retirado = depEletronicos.retirar();
if (retirado.isPresent()) {
    Eletronico item = retirado.get(); // Sem cast (Eletronico)
    System.out.println("Retirado com sucesso: " + item.getNome() + " (R$ "
+ item.getPreco() + ")");
}
```

3. Aplicação de Métodos Genéricos e Escolha dos Wildcards

Na classe `Deposito`, implementamos três operações utilizando métodos genéricos e *wildcards*, escolhendo cada um pelo seguinte motivo técnico (Tarefa 3.d):

Método Implementado	Recurso / Wildcard	Justificativa da Escolha
`buscarPorNome(Deposito<T> dep, String nome)`	Método genérico `<T extends Produto>` com retorno `Optional<T>`	Preserva o tipo exato `T` do depósito pesquisado. Se buscarmos em um `Deposito<Eletronico>`, o método retorna `Optional<Eletronico>` sem perder o subtipo específico.
`transferir(Deposito<? extends T> origem, Deposito<? super T> destino)`	Regra PECS: `<? extends T>` na origem e `<? super T>` no destino	O depósito de **origem** apenas fornece itens (*Producer* → `extends`), enquanto o depósito de **destino** recebe e guarda esses itens (*Consumer* → `super`). Isso permite transferir itens de um `Deposito<Eletronico>` para um `Deposito<Produto>` geral, algo que seria impossível sem wildcards devido à invariância.

Método Implementado	Recurso / Wildcard	Justificativa da Escolha
`listarConteudo(Deposito<?> deposito)`	Unbounded Wildcard `<?>`	O método apenas imprime os itens no console e não precisa inserir nada no depósito. Como `Deposito<T extends Produto>` já tem limite superior em `Produto`, `Deposito<?>` aceita qualquer depósito (`Eletronico`, `Alimento`, `Vestuario` ou `Produto`) com máxima simplicidade.

Saída da Execução da Solução Final

Resultado obtido no console ao executar a classe `Main` refatorada:

```
--- CONTEÚDO INICIAL DOS DEPÓSITOS ---
=== Depósito de Eletrônicos ===
- Eletrônico: Notebook Dell Inspiron (R$ 4200,00) - Garantia: 12 meses  -
  Eletrônico: Monitor LG 24pol (R$ 890,00) - Garantia: 24 meses
=== Depósito de Alimentos ===
- Alimento: Arroz Tipo 1 5kg (R$ 28,50) - Validade: 15/08/2027
- Alimento: Café Torrado 500g (R$ 19,90) - Validade: 10/05/2027
=== Depósito de Vestuário ===
- Vestuário: Camiseta Algodão (R$ 59,90) - Tamanho: G
- Vestuário: Jaqueta Jeans (R$ 189,90) - Tamanho: M

--- BUSCA POR NOME (MÉTODO GENÉRICO + OPTIONAL) ---
Encontrado: Notebook Dell Inspiron | Garantia: 12 meses

--- RETIRADA DE ITEM SEM NECESSIDADE DE CAST ---
Retirado com sucesso: Monitor LG 24pol (R$ 890.0)

--- TRANSFERÊNCIA COM WILDCARDS (REGRA PECS) --===
Depósito de Eletrônicos ===
  (Depósito vazio)
=== Depósito Geral de Expedição ===
- Eletrônico: Notebook Dell Inspiron (R$ 4200,00) - Garantia: 12 meses
- Alimento: Café Torrado 500g (R$ 19,90) - Validade: 10/05/2027
- Alimento: Arroz Tipo 1 5kg (R$ 28,50) - Validade: 15/08/2027
```

4. Instruções de Compilação e Declaração de Contribuição

O projeto foi desenvolvido utilizando o **Java JDK 17** e a IDE **IntelliJ IDEA**. Para compilar e executar no IntelliJ IDEA, basta abrir a pasta do projeto em *File > Open*, verificar se o Project SDK está definido como JDK 17 (ou superior) em *File > Project Structure*, clicar com

o botão direito sobre o arquivo `Main.java` e selecionar **Run 'Main.main()'**. Pelo terminal, basta executar `javac *.java` seguido de `java Main`.

Referências Bibliográficas

- ORACLE. **The Java™ Tutorials — Lesson: Generics (Updated)** (Why Use Generics, Generic Types, Raw Types, Generic Methods, Bounded Type Parameters, Wildcards e Type Erasure). Disponível em: https://docs.oracle.com/javase/tutorial/java/generics/index.html. Acesso em: nov. 2026.
- ORACLE. **Guidelines for Wildcard Use (In and Out Principle)**. Oracle Help Center. Disponível em: https://docs.oracle.com/javase/tutorial/java/generics/wildcardGuidelines.html. Acesso em: nov. 2026.
- BLOCH, Joshua. **Effective Java**. 3. ed. Boston: Addison-Wesley Professional, 2018. (Capítulo 5: *Generics* — Item 26: *Don't use raw types*; Item 28: *Prefer lists to arrays*; Item 31: *Use bounded wildcards to increase API flexibility / PECS*). Referência complementar disponível em: Capítulo 5 - Effective Java (Pearson).